

Справочник методов и классов приложения catalog

Django + DRF — Amazon Clone

Версия 1.0 · Июнь 2026

10 моделей · 2 сервиса · 1 queryset · 9 сериализаторов · 8 views

Ссылки на официальную документацию Django / DRF / treebeard

Содержание

1. Модели

- 1.1 Product
- 1.2 ProductVariant
- 1.3 ProductImage
- 1.4 Brand
- 1.5 Category (treebeard)
- 1.6 Tag
- 1.7 Attribute / AttributeValue / VariantAttribute (EAV)

2. Константы (ProductStatus)

3. QuerySet (ProductQuerySet)

4. Manager (ProductManager)

5. Сервисы

- 5.1 CatalogService
- 5.2 SlugService

6. Сериализаторы

- 6.1 Product: List / Detail / Query / Input
- 6.2 Brand: List / Detail
- 6.3 Category: Tree / Detail / Breadcrumb
- 6.4 Tag

7. Views (API-эндпоинты)

- 7.1 Product: List / Detail / Create / Update
- 7.2 Brand: List / Detail
- 7.3 Category: Tree / Detail

8. Сигналы (signals.py)

9. Admin

10. URL-маршруты

Приложение: Глоссарий

1. Модели

1.1 Product

Главная сущность каталога. Файл: `apps/catalog/models/product.py`.

Поля модели

Поле	Тип Django	Назначение
uuid	UUIDField	Публичный ID для API (скрывает последовательный PK)
name	CharField(255, db_index)	Название товара, индекс для точного поиска
slug	SlugField(unique, blank)	SEO-URL, автогенерация один раз при создании
description	TextField(blank)	Полное описание товара
search_vector	SearchVectorField(null)	PostgreSQL полнотекстовый поиск (tsvector + GIN)
meta_title	CharField(255, blank)	SEO <title>
meta_description	TextField(blank)	SEO <meta description>
manufacturer_code	CharField(100, db_index)	Артикул производителя
status	CharField(choices, db_index)	DRAFT / ACTIVE / OUT_OF_STOCK / ARCHIVED
is_featured	BooleanField(db_index)	Рекомендуемый товар
published_at	DateTimeField(db_index)	Дата/время публикации (отложенная публикация)
brand	FK(Brand, PROTECT)	Бренд товара. PROTECT — нельзя удалить бренд с товарами
categories	M2M(Category)	Категории (M2M — товар в нескольких)
primary_category	FK(Category, PROTECT)	Главная категория (для breadcrumbs + canonical URL)
tags	M2M(Tag)	Теги товара
main_image	FK(ProductImage, null)	Денормализованная ссылка на главное изображение
min_price	DecimalField(null, db_index)	Денормализованная мин. цена из вариантов
max_price	DecimalField(null)	Денормализованная макс. цена из вариантов
rating	DecimalField(3,2, db_index)	Средний рейтинг 0.00–5.00
reviews_count	PositiveIntegerField	Количество отзывов
views_count	PositiveBigIntegerField	Количество просмотров (64-bit)

Методы модели

```
save(self, *args, **kwargs) → None
```

Генерирует `slug` при первом сохранении, устанавливает `published_at` при переходе в ACTIVE.

[DOC] [Model.save\(\)](#) · [slugify\(\)](#)

```
increment_views(self) → None
```

Атомарно +1 к `views_count` через `F()`. Без `F()` — race condition при параллельных запросах.

```
self.views_count = F('views_count') + 1
self.save(update_fields=['views_count', 'updated_at'])
self.refresh_from_db(fields=['views_count'])
```

[DOC] [F\(\) expressions](#) · [refresh from db\(\)](#)

```
update_rating(self, new_rating: Decimal, total_reviews: int) → None
```

Пересчитывает рейтинг и количество отзывов. Вызывается из ReviewService.

```
recalculate_prices(self) → None
```

Пересчитывает `min_price` / `max_price` из активных вариантов. Использует lazy-импорт для избежания циклов.

Свойства (@property)

Свойство	Тип	Описание
<code>is_active</code>	bool	<code>status == ProductStatus.ACTIVE</code>
<code>is_visible</code>	bool	ACTIVE + <code>published_at</code> в прошлом или NULL
<code>display_rating</code>	str	Рейтинг для UI: «4.50»
<code>price_range</code>	str	«1 299.00 — 3 499.00» или «Цена не указана»

Meta-оптимизации

Partial indexes (условные индексы — только для ACTIVE товаров):

- `product_active_category_idx` — `(status, primary_category) WHERE status='active'`

- `product_active_brand_idx` — (status, brand) WHERE status='active'
- `product_top_rating_idx` — (-rating) WHERE status='active'
- `product_price_asc_idx` — (min_price) WHERE status='active'
- `product_price_desc_idx` — (-max_price) WHERE status='active'
- `product_newest_idx` — (-created_at) WHERE status='active'
- `product_featured_idx` — (-created_at) WHERE status='active' AND is_featured=True

GIN index: `product_search_gin` на `search_vector`

CheckConstraint: rating 0-5, счётчики ≥ 0 , $\text{min_price} \geq 0$, $\text{max_price} \geq \text{min_price}$

[DOC] [Conditional \(partial\) indexes](#) · [CheckConstraint](#) · [GinIndex](#) · [SearchVectorField](#)

1.2 ProductVariant

Конкретное исполнение товара. Файл: `apps/catalog/models/product_variant.py`.

Поле	Тип	Описание
<code>product</code>	FK(Product, CASCADE)	Товар-владелец варианта
<code>sku</code>	CharField(100, unique)	Складской артикул (IP15P-128-T)
<code>barcode</code>	CharField(100, db_index)	Штрихкод EAN-13 / UPC-A
<code>is_active</code>	BooleanField(db_index)	Видимость в каталоге
<code>length/width/height</code>	DecimalField(validators, null)	Габариты для доставки
<code>weight</code>	DecimalField(validators, null)	Вес для доставки
<code>slug</code>	SlugField(unique, null)	URL варианта (опционально)

Index: `variant_product_active_idx` на (product, is_active)

save(): автогенерация slug из «{product.name}-{sku}»

[DOC] [on_delete=CASCADE](#) · [MinValueValidator](#)

1.3 ProductImage

Изображение товара. Файл: `apps/catalog/models/product_image.py`.

Поле	Тип	Описание
product	FK(Product, CASCADE)	Товар
image	ImageField(upload_to='products/%Y/%m/')	Файл изображения
alt	CharField(255, blank)	SEO alt-текст
is_main	BooleanField	Главное изображение (только одно на товар)
order	PositiveIntegerField	Сортировка (drag-and-drop)

Partial UniqueConstraint: `unique_main_product_image` — `UNIQUE(product) WHERE is_main=True`

[DOC] [ImageField](#) · [UniqueConstraint\(condition=...\)](#)

1.4 Brand

Бренд товара. Файл: `apps/catalog/models/brand.py`.

Поле	Тип	Описание
name	CharField(100, unique)	Название бренда
slug	SlugField(255, unique, blank)	URL-slug, автогенерация, НЕ меняется при переименовании
description	TextField(blank)	Описание бренда
logo	ImageField(upload_to='brands/%Y/%m/', null)	Логотип
is_active	BooleanField(db_index)	Мягкое удаление

```
get_absolute_url(self) → str
```

Возвращает `/brands/{self.slug}/`

[DOC] [get_absolute_url\(\)](#)

1.5 Category (treebeard MP_Node)

Иерархическая категория. Файл: `apps/catalog/models/category.py`. Наследует `MP_Node` + `BaseModel`.

Поле	Тип	Описание
name	CharField(200)	Название категории
slug	SlugField(unique, null)	URL-slug
description	TextField(blank)	Описание
image	ImageField(null)	Изображение
is_active	BooleanField(db_index)	Активна ли
meta_title	CharField(255, blank)	SEO title
meta_description	TextField(blank)	SEO description
url_path	CharField(1000, editable=False, db_index)	Денормализованный: 'electronics/phones/smartphones'
full_name_cached	CharField(2000, editable=False)	Денормализованный: 'Электроника → Телефоны → Смартфоны'

Методы treebeard (от MP_Node)

Метод	Описание	Документация
Category.add_root(name=...)	Создать корневую категорию	Adding nodes
node.add_child(name=...)	Создать дочернюю категорию	Adding nodes
node.move(target, pos)	Переместить узел	Moving nodes
node.get_children()	Прямые потомки (QuerySet)	get_children()
node.get_ancestors()	Все предки (один SQL-запрос)	get_ancestors()
node.get_descendants()	Все потомки (один SQL-запрос)	get_descendants()
node.get_parent()	Родительский узел	get_parent()
node.is_root()	Является ли корневым узлом	is_root()
Category.get_root_nodes()	Все корневые категории	get_root_nodes()

Кастомные методы

```
_build_url_path(self) → str
```

Склеивает slug'и всех предков через '/'

```
_build_full_name(self) → str
```

Склеивает названия предков через ' → '


```
_refresh_descendants(self) → None
```

Каскадно пересчитывает `url_path` и `full_name_cached` у BCEX потомков через `bulk_update(batch_size=500)`

```
_rebuild_caches(self)
```

Пересчитывает кэш для одного узла

Запрещено!

`Category.objects.create(...)` — `treebeard` не обновит дерево. Только `add_root()` / `add_child()`

[DOC] [django-treebeard MP Node](#) · [bulk_update\(\)](#)

1.6 Tag

Плоская метка товара. Файл: `apps/catalog/models/tag.py`.

Поле	Тип	Описание
<code>name</code>	<code>CharField(100, unique)</code>	Название тега
<code>slug</code>	<code>SlugField(100, unique, blank)</code>	URL-slug, автогенерация
<code>is_active</code>	<code>BooleanField(db_index)</code>	Активен

1.7 EAV: Attribute / AttributeValue / VariantAttribute

Attribute

Название характеристики: «Цвет», «Размер», «Память». Файл: `apps/catalog/models/attribute.py`.

Поле	Тип	Описание
<code>name</code>	<code>CharField(100, unique)</code>	Название атрибута
<code>slug</code>	<code>SlugField(100, unique, blank)</code>	URL-slug
<code>description</code>	<code>TextField(blank)</code>	Пояснение для менеджеров

AttributeValue

Допустимое значение атрибута: «Красный», «XL», «256 GB». Файл: `apps/catalog/models/attribute_value.py`.

Поле	Тип	Описание
<code>attribute</code>	<code>FK(Attribute, CASCADE)</code>	Принадлежность атрибуту
<code>value</code>	<code>CharField(100)</code>	Текстовое значение
<code>color_hex</code>	<code>CharField(7, blank)</code>	HEX-цвет для UI (<code>#FF0000</code>)

UniqueConstraint: (`attribute`, `value`) — у атрибута «Цвет» не может быть двух «Красный»

VariantAttribute

Связь варианта со значением атрибута. Файл: `apps/catalog/models/variant_attribute.py`.

Поле	Тип	Описание
<code>variant</code>	<code>FK(ProductVariant, CASCADE)</code>	Вариант товара
<code>attribute</code>	<code>FK(Attribute, CASCADE)</code>	Атрибут
<code>value</code>	<code>FK(AttributeValue, CASCADE)</code>	Значение

UniqueConstraint: (`variant`, `attribute`) — у варианта только ОДНО значение каждого атрибута

clean(): проверяет что `value.attribute_id == attribute_id` (нельзя Цвет=XL)

[DOC] [Model.clean\(\)](#) · [EAV pattern \(Wikipedia\)](#)

2. Константы (ProductStatus)

Файл: `apps/catalog/constants.py`. `TextChoices` — Django-аналог Enum.

```
class ProductStatus(models.TextChoices):
    DRAFT = 'draft', 'Черновик'
    ACTIVE = 'active', 'Активен'
    OUT_OF_STOCK = 'out_of_stock', 'Нет в наличии'
    ARCHIVED = 'archived', 'Архив'
```

```
is_visible_in_catalog (property) → bool
```

Только `ACTIVE` виден покупателям.

[DOC] [TextChoices](#) · [get_FOO_display\(\)](#)

3. QuerySet (ProductQuerySet)

Файл: `apps/catalog/querysets/product_queryset.py`. Компонуемый QuerySet: каждый метод возвращает новый QuerySet.

Фильтрация

Метод	SQL-аналог	Описание
<code>.active()</code>	<code>WHERE status='active'</code>	Только активные товары
<code>.visible()</code>	<code>WHERE status='active' AND primary_category__is_active AND (published_at IS NULL OR published_at <= NOW)</code>	Видимые в каталоге
<code>.featured()</code>	<code>WHERE status='active' AND is_featured=True</code>	Рекомендуемые
<code>.for_category(cat)</code>	<code>JOIN product_categories WHERE category_id=X</code>	Товары в категории (M2M)
<code>.for_brand(brand)</code>	<code>WHERE brand_id=X</code>	Товары бренда
<code>.price_range(min, max)</code>	<code>WHERE min_price >= X AND max_price <= Y</code>	По денормализованным ценам
<code>.search(query)</code>	<code>WHERE search_vector @@ to_tsquery(query)</code> (PostgreSQL) или <code>ILIKE</code> (SQLite)	Полнотекстовый поиск

Оптимизация (prefetch / select_related)

Метод	Что делает	Экономия запросов
<code>.with_related()</code>	<code>select_related('brand', 'primary_category', 'main_image')</code>	3 JOIN вместо N+1
<code>.with_images()</code>	<code>prefetch_related('images')</code>	1 запрос вместо N
<code>.with_variants()</code>	<code>Prefetch('variants', queryset=активные_с_ценой_и_стоком)</code>	1 запрос + JOIN вместо 2N
<code>.with_categories()</code>	<code>prefetch_related('categories')</code>	1 запрос вместо N

Композитные методы

Метод	Цепочка	Использование
<code>.catalog()</code>	<code>visible + with_related + with_images + with_variants</code>	Listing-страница каталога
<code>.for_card()</code>	<code>with_related + with_images + with_variants + with_categories</code>	Карточка товара (detail)

[DOC] [select_related\(\)](#) · [prefetch_related\(\)](#) · [Prefetch object](#) · [Q objects](#) · [PostgreSQL full-text search](#)

4. Manager (ProductManager)

Файл: `apps/catalog/managers/product_manager.py`.

```
class ProductManager(models.Manager.from_queryset(ProductQuerySet)):
    pass
```

`from_queryset()` автоматически переносит все методы `QuerySet` в `Manager`.

Результат: `Product.objects.active()`, `Product.objects.catalog()` и т.д.

[DOC] [Manager.from_queryset\(\)](#) · [Calling custom QuerySet methods](#)

5. Сервисы

5.1 CatalogService

Файл: `apps/catalog/services/catalog_service.py`. Все методы — `@staticmethod`.

Товары

Метод	Описание	Возвращает
<code>get_product_by_uuid(uuid)</code>	Поиск по UUID с <code>for_card()</code>	Product / NotFound
<code>get_product_by_slug(slug)</code>	Поиск по slug с <code>for_card()</code>	Product / NotFound
<code>get_product_listing(...)</code>	Listing с фильтрами + пагинация	(QuerySet, applied_filters)
<code>create_product(...)</code>	Создание товара в транзакции	Product
<code>update_product(product, ...)</code>	Обновление товара с валидацией	Product
<code>increment_product_views(product)</code>	F()-инкремент просмотров	None

Категории

Метод	Описание
<code>get_category_tree()</code>	<code>Category.get_root_nodes()</code> — корневые категории
<code>get_category_by_slug(slug)</code>	Категория по slug / NotFound
<code>get_category_breadcrumbs(category)</code>	Цепочка предков + сама категория

Бренды / Теги

Метод	Описание
<code>get_active_brands()</code>	<code>Brand.objects.filter(is_active=True).order_by('name')</code>
<code>get_brand_by_slug(slug)</code>	Бренд по slug / NotFound
<code>get_active_tags()</code>	<code>Tag.objects.filter(is_active=True).order_by('name')</code>

```
get_product_listing(*, category_slug, brand_slug, tag_slug, min_price,
max_price, search_query, ordering) → tuple[QuerySet, dict]
```

* — keyword-only аргументы. Возвращает (qs, applied_filters) для UI.

```
create_product(*, name, brand_id, ...) → Product
```

Обёрнут в `@transaction.atomic`. Создаёт товар + M2M-связи.

Паттерн: View → Serializer → Service → ORM

View не содержит бизнес-логику. View вызывает Service, Service работает с ORM. Это позволяет переиспользовать Service из management-команд, Celery-задач, тестов.

[DOC] [transaction.atomic](#) · [get_object_or_404\(\)](#) · [NotFound \(DRF\)](#)

5.2 SlugService

Файл: `apps/catalog/services/slug_service.py`.

```
generate_unique_slug(instance, field_value, slug_field_name='slug') → str
```

Генерирует уникальный slug. Если занят — добавляет суффикс: «nike» → «nike-2» → «nike-3».

Использует `django.utils.text.slugify()`. Исключает текущий объект из проверки (`exclude(pk=instance.pk)`).

[DOC] [slugify\(\)](#)

6. Сериализаторы

6.1 Product сериализаторы

Файл: `apps/catalog/serializers/product_serializers.py`.

Класс	Тип	Назначение	Поля
ProductImageSerializer	ModelSerializer	Изображение (только чтение)	id, image, alt, is_main, order
ProductVariantListSerializer	ModelSerializer	Вариант для карточки	id, sku, slug, price, stock_quantity, is_active
ProductListSerializer	ModelSerializer	Listing (50 товаров)	uuid, name, slug, brand_*, category_*, main_image_url, prices, rating, ...
ProductDetailSerializer	ModelSerializer	Карточка товара	Bce + description, images, variants, tags, SEO
ProductListQuerySerializer	Serializer	Валидация ? category=&brand=&search=	category, brand, tag, min_price, max_price, search, ordering
CreateProductInputSerializer	Serializer	Валидация POST-тела	name, brand_id, primary_category_id, description, ...
UpdateProductInputSerializer	Serializer	Валидация PATCH-тела	Все поля optional

Ключевые DRF-классы и паттерны

Класс/атрибут	Применение	Документация
serializers.ModelSerializer	Авто-поля из модели	ModelSerializer
serializers.Serializer	Ручные поля (input/query)	Serializer
source='brand.name'	Навигация по FK	source
SlugRelatedField	Отображать slug вместо id	SlugRelatedField
SerializerMethodField	Вычисляемое поле (get_*)	SerializerMethodField
read_only_fields = fields	Все поля только для чтения	read_only_fields
ImageField	Авто-URL из ImageField модели	ImageField (DRF)

6.2 Brand сериализаторы

Файл: `apps/catalog/serializers/brand_serializers.py`.

Класс	Поля
BrandListSerializer	id, name, slug, logo
BrandDetailSerializer	+, description, products_count (динамическое)

6.3 Category сериализаторы

Файл: `apps/catalog/serializers/category_serializers.py`.

Класс	Поля
BreadcrumbSerializer	name, slug, url_path
CategoryTreeSerializer	id, name, slug, url_path, depth, is_active, children (SerializerMethodField — рекурсия!)
CategoryDetailSerializer	+, description, image, full_name_cached, breadcrumbs, products_count

6.4 Tag сериализатор

Файл: `apps/catalog/serializers/tag_serializers.py`.

Класс	Поля
TagSerializer	id, name, slug (все read-only)

7. Views (API-эндпоинты)

7.1 Product Views

Файл: `apps/catalog/api_views/product_views.py`.

View	Метод	URL	Permissions
ProductListView	GET	/api/v1/catalog/products/	AllowAny
ProductDetailView	GET	/api/v1/catalog/products/{id}/	AllowAny
ProductCreateView	POST	/api/v1/catalog/products/create/	IsAuthenticated
ProductUpdateView	PATCH	/api/v1/catalog/products/{uuid}/update/	IsAuthenticated

Ключевые DRF-классы

Класс	Применение	Документация
<code>APIView</code>	Базовый класс для всех views	APIView
<code>AllowAny</code>	Публичный доступ (без JWT)	AllowAny
<code>IsAuthenticated</code>	Требует JWT	IsAuthenticated
<code>Response(data)</code>	JSON-ответ	Response
<code>PageNumberPagination</code>	Пагинация listings	PageNumberPagination
<code>extend_schema</code>	Swagger-документация	extend_schema

7.2 Brand Views

View	Метод	URL	Описание
<code>BrandListView</code>	GET	<code>/api/v1/catalog/brands/</code>	Все активные бренды
<code>BrandDetailView</code>	GET	<code>/api/v1/catalog/brands/{slug}/</code>	Бренд + <code>products_count</code> (<code>setattr</code>)

7.3 Category Views

View	Метод	URL	Описание
<code>CategoryTreeView</code>	GET	<code>/api/v1/catalog/categories/</code>	Рекурсивное дерево
<code>CategoryDetailView</code>	GET	<code>/api/v1/catalog/categories/{slug}/</code>	Категория + breadcrumbs + <code>products_count</code>

8. Сигналы (signals.py)

Файл: `apps/catalog/signals.py`.

Сигнал	Sender	Действие
<code>post_save(ProductImage)</code>	<code>ProductImage</code>	Установка/сброс <code>Product.main_image</code>
<code>post_delete(ProductImage)</code>	<code>ProductImage</code>	Очистка <code>Product.main_image</code> при удалении
<code>post_save(ProductVariant)</code>	<code>ProductVariant</code>	Пересчёт <code>min_price/max_price</code> при изменении <code>is_active</code>
<code>post_delete(ProductVariant)</code>	<code>ProductVariant</code>	Пересчёт <code>min_price/max_price</code>
<code>post_save(Product)</code>	<code>Product</code>	Обновление <code>search_vector</code> (только при изменении <code>name/description</code>)

Ключевые методы в сигналах

```
update_product_main_image(sender, instance, **kwargs)
```

При `is_main=True`: `Product.objects.filter(pk=...).update(main_image=image)` — обходит сигналы `Product` (оптимально).

```
clear_product_main_image_on_delete(sender, instance, **kwargs)
```

При удалении главного изображения:

```
Product.objects.filter(main_image=instance).update(main_image=None)
```

```
on_variant_change(sender, instance, **kwargs)
```

Если `is_active` изменился → пересчитывает цены товара. Проверяет

`kwargs['created']` — только при обновлении, не создании.

```
update_product_search_vector(sender, instance, **kwargs)
```

Обновляет `search_vector` через `SearchVector('name', weight='A', config='russian') + SearchVector('description', weight='B', config='russian')`. Пропускается если `update_fields` не включает `name/description`. SQLite-совместимо (пропускает тихо).

[DOC] [post_save](#) · [post_delete](#) · [SearchVector](#) · [QuerySet.update\(\)](#)

9. Admin

Admin-класс	Модель	Особенности
ProductAdmin	Product	Fieldsets (6 секций), inline-изображения/варианты, цветной статус, price_range
CategoryAdmin	Category	treebeard поля в collapse-секции, actions (activate/deactivate), url_path
BrandAdmin	Brand	prepopulated_fields={'slug': ('name',)}, logo_preview
TagAdmin	Tag	list_editable=('is_active',), slug readonly
ProductVariantAdmin	ProductVariant	list_filter по is_active, поиск по SKU
AttributeAdmin	Attribute	AttributeValueInline, values_count
AttributeValueAdmin	AttributeValue	color_hex preview, list_filter по attribute

Ключевые Admin-классы и методы

Класс/метод	Применение	Документация
admin.ModelAdmin	Базовый класс для admin	ModelAdmin options
admin.register(Model)	Декоратор регистрации	@admin.register()
TabularInline	Inline-таблица (изображения, варианты)	TabularInline
fieldsets	Группировка полей	fieldsets
prepopulated_fields	Автогенерация slug из name	prepopulated_fields
list_editable	Inline-редактирование в списке	list_editable
format_html()	Безопасный HTML в admin (превью, цвета)	format_html()
list_select_related	Оптимизация списка (JOIN)	list_select_related
@admin.action	Массовые действия	Admin actions
admin.display(description=...)	Кастомные колонки	admin.display

10. URL-маршруты

Файл: `apps/catalog/urls.py`.

Метод	URL	View	Описание
GET	/api/v1/catalog/products/	ProductListView	Listing с фильтрами
GET	/api/v1/catalog/products/{slug_or_uuid}/	ProductDetailView	Карточка товара
POST	/api/v1/catalog/products/create/	ProductCreateView	Создание (staff)
PATCH	/api/v1/catalog/products/{uuid}/update/	ProductUpdateView	Обновление (staff)
GET	/api/v1/catalog/categories/	CategoryTreeView	Дерево категорий
GET	/api/v1/catalog/categories/{slug}/	CategoryDetailView	Детали категории
GET	/api/v1/catalog/brands/	BrandListView	Список брендов
GET	/api/v1/catalog/brands/{slug}/	BrandDetailView	Детали бренда

[DOC] [Django URL dispatcher](#) · [path\(\)](#) · [include\(\)](#)

Приложение: Глоссарий

Термин	Определение	Документация
BaseModel	Абстрактная модель с id (BigAutoField), created_at, updated_at	Abstract base classes
UUIDField	Хранит UUID v4 (случайный 128-битный ID)	UUIDField
SlugField	URL-friendly текст: только a-z, 0-9, дефисы	SlugField
PROTECT	Запрещает удаление объекта, на который ссылаются	on_delete
CASCADE	При удалении родителя — удалить всех детей	on_delete
ManyToManyField	M2M связь через промежуточную таблицу	ManyToManyField
SearchVectorField	PostgreSQL tsvector для полнотекстового поиска	SearchVectorField
GinIndex	Инвертированный индекс (для tsvector)	GinIndex
F() expression	Ссылка на значение поля в БД (атомарная арифметика)	F()
Q object	OR/AND/NOT условия для filter()	Q objects
Prefetch	Кастомный QuerySet для prefetch_related	Prefetch
select_related	JOIN для FK/OneToOne (1 запрос)	select_related
prefetch_related	Отдельный SELECT для M2M/reverse FK (2 запроса)	prefetch_related
bulk_update	Обновление многих объектов одним SQL	bulk_update
MP_Node (treebeard)	Materialized Path: хранение дерева в БД	MP_Node docs
EAV	Entity-Attribute-Value: гибкие характеристики товаров	EAV pattern
UniqueConstraint(condition)	Частичный уникальный индекс (WHERE ...)	UniqueConstraint
CheckConstraint	Ограничение-проверка на уровне БД	CheckConstraint
Partial index	Индекс с условием WHERE (только ACTIVE строки)	Conditional indexes
TextChoices	Django-Enum для CharField choices	TextChoices

ImageField	Поле для загрузки файлов-изображений	ImageField
transaction.atomic	«Всё или ничего»: BEGIN...COMMIT/ROLLBACK	atomic
ModelSerializer	DRF-сериализатор, автогенерация полей из модели	ModelSerializer
APIView	DRF-базовый класс для API views	APIView
PageNumberPagination	Страничная пагинация DRF	Pagination
AllowAny / IsAuthenticated	Классы разрешений DRF	Permissions
from_queryset()	Магия: методы QS доступны через Manager	from_queryset
drf-spectacular	Генерация OpenAPI/Swagger из DRF views	drf-spectacular